

Quantitative Enhancements for Prediction Market Trading Algorithms: A Strategic Implementation Report

The operational metrics extracted from the current algorithmic trading system—specifically 72 closed paper trades yielding a positive realized P&L of \$17 alongside a 42% win rate—reveal critical data regarding the underlying strategy's statistical expectancy and structural vulnerabilities. In the context of binary prediction markets, a 42% win rate implies that the algorithm is entirely reliant on capturing asymmetric risk-reward ratios to maintain its marginal profitability. While the system demonstrates a slight positive mathematical expectation in a paper-trading environment, this profile is highly sensitive to variance, parameter uncertainty, and terminal event volatility. In live execution, bid-ask spreads, gas fees, and slippage will rapidly consume a \$17 theoretical edge unless the execution architecture is rigorously optimized.

Operating under the constraints of a Python-based infrastructure, n8n workflow automation, Claude Code integration, and a strict API budget of under \$50 per month, the development focus must permanently pivot away from latency-sensitive operations. Holding periods of 24 to 48 hours dictate a swing-trading or short-term position-trading paradigm, rendering pure microsecond arbitrage obsolete for this architecture. Instead, alpha must be extracted through combinatorial pricing discrepancies, rigorous volume filtering, dynamic risk management protocols, and the integration of high-quality, low-cost external data oracles. This report outlines the necessary quantitative and infrastructural enhancements required to transition the current paper-trading logic into a robust, capital-preserving, and scalable system.

1. Redefining Arbitrage Opportunities in 2026

The theoretical foundation of arbitrage in decentralized prediction markets remains mathematically sound: risk-free profit is achieved when the aggregate cost of mutually exclusive outcomes across single or multiple venues falls below 100% implied probability (or \$1.00).¹ However, the execution environment has undergone a drastic evolutionary shift, requiring retail algorithms to abandon pure speed in favor of structural intelligence.

1.1 The Collapse of Pure Cross-Exchange Arbitrage

Historically, retail Python algorithms could exploit direct pricing discrepancies between platforms such as Polymarket (Polygon-based, crypto settlement), Kalshi (fiat-settled, CFTC regulated), and Manifold Markets (play-money "Mana").³ Recent empirical analyses of order book data, however, reveal that the average duration of a pure arbitrage opportunity has

compressed from 12.3 seconds in 2024 to merely 2.7 seconds by the first quarter of 2026.⁵

High-frequency trading (HFT) infrastructure, operating on dedicated Polygon Remote Procedure Call (RPC) nodes with sub-100 millisecond latency, now captures approximately 73% of these transient spreads.⁵ Sophisticated quantitative funds extract millions by applying advanced mathematical frameworks, such as Bregman projections and Frank-Wolfe algorithms, to execute multi-leg trades instantaneously.⁷ Furthermore, the median arbitrage spread currently sits at a marginal 0.3%, a margin that is entirely consumed by execution friction and slippage during slightly delayed API calls.⁵ For a system limited by standard REST API rate limits and n8n polling frequencies, competing in this execution tier is mathematically unviable.

1.2 Combinatorial and Dutch Book Strategies

Rather than racing HFT bots for transient cross-platform spreads, a 24-to-48-hour algorithmic strategy should target combinatorial discrepancies, commonly referred to as Dutch book arbitrage.² Because decentralized prediction markets like Polymarket do not allow native shorting or margin leverage, capital must be efficiently allocated to buy corresponding "Yes" shares across divergent, yet correlated, contracts.²

A highly effective avenue for a Python-based bot is exploiting the correlation between related markets within the same platform. When news breaks, liquidity flows unevenly across these related derivative contracts, creating temporary pricing dislocations. The algorithm can execute a combinatorial strategy by purchasing mispriced legs across these nested arrays. However, it is critical to ensure that the underlying resolution oracles (such as the UMA Optimistic Oracle on Polymarket) are identical across the selected markets to prevent non-atomic execution risks, where one leg resolves favorably and the other is disputed.⁴

1.3 Sports Betting Integrations and +EV Polling

By late 2025, sports contracts constituted over 60% of open interest on Polymarket, drastically altering the liquidity landscape.⁸ This concentration presents a unique opportunity to utilize traditional, sharp sportsbooks as a predictive oracle for prediction market pricing.

The integration of sports betting odds APIs provides a critical advantage for short-term swing trading.⁹ "The Odds API" is highly recommended for this architecture, as it offers a free tier of 500 requests per month, perfectly aligning with the <\$50/month budgetary constraint.¹⁰ Python scripts can periodically poll The Odds API to aggregate live odds from sharp international bookmakers (e.g., Pinnacle) and compare these implied probabilities against the current Central Limit Order Book (CLOB) midpoint on Polymarket.¹⁰

When a sharp sportsbook calculates an implied probability of 65% for an event, but the corresponding Polymarket contract is trading at \$0.55 (55% implied probability), a +EV

(Expected Value) opportunity is identified. Open-source Python repositories, such as arbitrage-scanner and its updated fork edge-scanner, have established frameworks for this exact cross-referencing logic.¹⁰ These repositories include pre-built functions for comparing soft book odds to Pinnacle's sharp lines, calculating the edge percentage, and automatically sizing the stake.¹⁰

Arbitrage Category	Target Discrepancy	Tool / API Requirement	Viability for 24-48h Strategy
Pure Cross-Platform	Polymarket vs. Kalshi direct price differences.	WebSocket feeds, dedicated RPC nodes, Rust backend.	Low. Spreads (0.3%) captured by HFT bots in <2.7 seconds. ⁵
Combinatorial / Dutch	Mispricing between nested markets (e.g., Game Winner vs. Spread).	Polymarket CLOB API, Python nested array parsing.	Medium. Requires identical resolution oracles to avoid atomic risk. ²
Sports +EV Polling	Pinnacle sharp odds vs. Polymarket CLOB midpoint.	The Odds API (Free Tier), Python Edge Scanner repo.	High. Exploits human mispricing on Polymarket without requiring microsecond speed. ¹⁰

2. Low-Risk Signal Enhancements and Data Ingestion

Operating a predictive algorithm with a sub-50% win rate mandates strict pre-trade filtering mechanisms. The system must systematically eliminate trades in illiquid markets where bid-ask spreads will instantly erode the realized P&L, and it must utilize high-conviction external signals to filter out false positives generated by basic technical analysis.

2.1 Enforcing Strict Liquidity and Volume Filters via the Gamma API

Order book depth varies exponentially across prediction markets. Popular political or macroeconomic events feature high liquidity and tight spreads, whereas niche contracts suffer from severe illiquidity and erratic pricing.¹¹ Placing automated market orders in illiquid environments results in catastrophic slippage, which is fatal for a system already exhibiting a 42% win rate.

To resolve this without heavy computational overhead, the Python script must query Polymarket's Gamma API rather than the CLOB API for initial market discovery. While the CLOB API is designed for active trading and order book management, the Gamma API provides rich metadata filtering capabilities required for safe market selection.¹² By structuring HTTP GET requests to <https://gamma-api.polymarket.com/events>, the algorithm can pass specific query parameters to eliminate untradable markets before any complex predictive logic is executed.¹²

The Python requests logic should enforce the following parameters:

- active=true and closed=false to ensure only live, tradable events are returned.
- order=volume_24hr combined with ascending=false to sort the data array by the highest recent trading volume.¹²
- Hard-coded Python filters to discard any market where the 24-hour volume falls below a defined threshold (e.g., \$50,000) or where the spread between the best bid and best ask exceeds 2%.

2.2 Ingesting Metaculus and Superforecaster Probabilities

Prediction market prices represent the aggregate sentiment of capital, which is frequently distorted by irrational retail flow, short-term news spikes, or manipulation.⁴ To gain an informational edge, the algorithm should cross-reference CLOB prices with the objective probabilities generated by professional forecasters.

Metaculus serves as the premier platform for rigorous, time-tested forecasting data, relying on proper scoring rules (such as log-based peer scoring) to extract true beliefs from participants.¹³ In early 2026, Metaculus launched comprehensive AI benchmarking tournaments, resulting in the release of robust Python tools for interacting with their datasets.¹³ The official forecasting-tools Python repository provides a direct API wrapper to pull aggregate community predictions and data from verified Superforecasters.¹³

The integration architecture is designed to validate trading signals:

1. The Python bot identifies a liquid target market on Polymarket trading at \$0.40 (40% implied probability).
2. The bot uses the forecasting-tools library to query Metaculus for the corresponding question.¹⁶
3. The script extracts the Metaculus Superforecaster median probability. If the Superforecaster median is 65%, a severe market dislocation is identified.¹⁷
4. The algorithm executes a position only when the delta between the prediction market price and the Superforecaster median exceeds a predefined confidence threshold (e.g., >15% variance).

This methodology effectively outsources complex predictive modeling to an established

network of experts, requiring minimal API calls and keeping costs effectively at zero.¹⁸

2.3 Cost-Effective Real-Time News and Sentiment Aggregation

Monitoring breaking news is vital for 24-to-48-hour holding periods, as market sentiment can shift violently based on external events. Traditional data firehoses, such as the enterprise Twitter/X API or Bloomberg terminals, far exceed the \$50 monthly constraint. However, alternative news aggregator APIs provide sufficient latency for swing-trading strategies and can be seamlessly integrated into n8n workflows.

News API Provider	Pricing Model	Best Use Case for Prediction Bots
NewsAPI.ai	Free tier: 2,000 searches/month.	Broad macroeconomic searches and scheduled 30-minute polling. ²⁰
GNews API	Free tier: 100 requests/day.	Focused querying for specific event-driven markets. ²⁰
DuckDuckGo (via SerpApi)	Free tier available.	Privacy-focused scraping for niche market titles. ²¹
ScrapingBee News API	Starts at \$49/month.	Real-time global news aggregation if the budget is maximized. ²¹

For real-time social sentiment, community developers utilize open APIs within n8n to aggregate Reddit and Quora discussions. By passing these textual snippets into a lightweight Large Language Model (LLM) node—such as Claude 3 Haiku or GPT-4o-mini—the workflow can generate a numerical "Sentiment Score".²² If the LLM determines that public sentiment on social forums is 90% bullish, but the Polymarket probability sits at 60%, the bot identifies an information arbitrage opportunity and triggers an entry signal.²²

3. Exit Optimization and Dynamic Risk Management

The survival of an algorithm with a 42% win rate depends entirely on mathematically sound position sizing and highly optimized exit protocols. In binary event derivatives, risk scales non-linearly as time to expiration decreases, requiring dynamic hedging and trailing stops rather than static take-profit levels.

3.1 Volatility-Adjusted Position Sizing: The Fractional Kelly Criterion

Standard fixed-fraction position sizing is fundamentally flawed for dynamic probability markets. The algorithm must dynamically scale capital allocation based on the calculated edge of each specific trade. The optimal mathematical framework for this is the Kelly Criterion, which maximizes the geometric growth rate of a portfolio over infinite trials.²³

For independent binary outcomes where the payoff is deterministic (\$1.00 for a correct prediction, \$0.00 for an incorrect prediction), the Kelly fraction f^* is calculated using the formula:

$$f^* = \frac{p(b + 1) - 1}{b}$$

Where p is the algorithm's estimated probability of winning (derived from the Superforecaster median or The Odds API), and b is the net fractional odds received on the wager, derived from the contract purchase price.²⁴

However, pure Kelly sizing requires the estimated probability p to be perfectly accurate. Prediction market parameters inherently contain noise, model uncertainty, and liquidity gaps.

Overestimating p leads to aggressive over-betting, which guarantees exponential capital destruction—a critical vulnerability noted by quantitative practitioners.²⁵

To implement this safely within the Python script, developers must utilize a "Fractional Kelly" calculator. By applying a dampening multiplier (typically 0.25 to 0.50, known as Quarter-Kelly or Half-Kelly) to the final Kelly fraction, the algorithm achieves near-optimal long-term growth while drastically reducing portfolio drawdown and variance.²⁶ If the Fractional Kelly output is negative, the script must systematically reject the trade, regardless of other technical indicators.²⁵ Furthermore, modern quantitative funds utilize "Universal Portfolio" algorithms that extend Kelly concepts to calculate weights for multiple simultaneous positions, ensuring the overall portfolio does not exceed risk parameters when holding 5-10 markets concurrently.²⁸

3.2 Dynamic Hedging and Time-Decay (Theta) Models

In traditional equity options, Theta represents the daily erosion of an option's extrinsic value as expiration approaches.³⁰ Binary prediction contracts exhibit unique time-decay mechanics. The prices do not merely reflect the probability of the event occurring; they reflect the *time remaining* for contrary information to emerge.³²

As the resolution date nears, the Gamma of a binary contract—the rate of change of the price

relative to incoming information—spikes exponentially.²⁷ A market priced at \$0.80 with two weeks remaining might absorb a negative news cycle and drop to \$0.60. The exact same news cycle with two hours remaining will instantly crash the price to \$0.05. This terminal risk concentration destroys algorithms that attempt to hold volatile contracts to final resolution.²⁷

The Python infrastructure must implement a time-decay exit model. A standard professional dynamic hedging protocol reduces position exposure as a function of the square root of time remaining²⁷:

$$Position(t) = Initial_Position \times \sqrt{\frac{T_{remaining}}{T_{initial}}}$$

By implementing this formula, the algorithm will incrementally scale out of the position as the event horizon approaches. For a 24-48 hour holding strategy, the bot should be programmed to hard-exit all positions prior to the final 10% of the contract's lifespan, entirely avoiding binary terminal volatility and locking in pre-resolution P&L.⁴

3.3 Trailing Stop-Loss (TSL) Implementations

To protect the current \$17 realized P&L and ensure gains are not surrendered to sudden market reversals, a trailing stop-loss (TSL) is mandatory. A TSL dynamically adjusts the exit trigger upward as the contract price appreciates, locking in profits while allowing the position to capture continued upside momentum.³⁵

Because decentralized prediction market APIs (like Polymarket's CLOB) do not typically support native conditional trailing orders on the exchange level, this logic must be maintained locally within the Python script.

1. Upon entry, the script records the entry_price and sets a highest_price variable equal to the entry.
2. The script polls the CLOB API periodically (e.g., every 5 minutes) via a cron job or n8n trigger.
3. If the current bid price exceeds the highest_price, the variable is updated in local memory (or a lightweight SQLite database).
4. If the current price falls below highest_price * (1 - trailing_percent), the script executes a REST API POST to the /orders endpoint to market-sell the position.¹¹

Existing open-source Python architectures, such as the BinanceAlert trailing stop-loss script, provide easily adaptable template logic. This script specifically calculates the percentage difference between the buying price and the current price, adjusting the stop loss dynamically without requiring constant user interaction.³⁷ Claude Code can easily refactor this logic to utilize the py-clob-client.

4. Automation Architecture: Python, n8n, and Claude Code Integration

Scaling the trading logic requires seamless interplay between the n8n orchestration layer, the Python execution scripts, and the external APIs. The open-source ecosystem for prediction markets in 2025 and 2026 has expanded rapidly, providing robust tools to accelerate this integration.³⁸

4.1 Native API Clients and GitHub Repositories (2025-2026)

The foundational tool for interaction is the official `py-clob-client`. Actively maintained by Polymarket, this Python library handles Layer 2 authentication (HMAC-SHA256 signatures), order creation, and WebSocket streaming.¹¹ Utilizing this official library guarantees compatibility with contract structure changes and reduces execution errors.³⁹

For Manifold Markets, the API is entirely open and operates on "Mana" (play money), making it the optimal sandbox for testing Kelly Criterion adjustments and Theta decay models before migrating algorithms to USDC-collateralized Polymarket.³ The `manifoldpy` wrapper abstracts the REST endpoints, allowing Python scripts to effortlessly pull probability histories and execute paper bets.⁴¹

To feed Claude Code optimal context for script generation, developers should reference recent, active repositories:

- **discountry/polymarket-trading-bot**: A highly relevant Python repository offering quickstart templates for executing trades, managing API keys, and deploying strategies.⁴² Passing this repository's file structure into Claude Code will yield highly accurate, modern execution scripts.
- **Panca2341/polymarket-trading-bot**: Provides robust examples of maintaining encrypted private keys and handling WebSocket live order book updates.⁴³
- **OpenClaw Framework**: OpenClaw is an autonomous AI agent framework designed to interface between decentralized prediction markets and central liquidity hubs.⁴⁴ While full deployment of OpenClaw requires dedicated hardware and significant capital (upwards of \$5,000)⁴⁵, its GitHub repository and the community-built "Polyclaw" skills offer exceptional architectural examples.⁴⁵ Analyzing the OpenClaw codebase provides best practices for separating hard-coded risk management protocols from LLM-driven filtering logic.⁴⁵

4.2 Enhancing the n8n Workflow Layer

n8n excels at orchestrating data pipelines without requiring continuous Python script loops. By utilizing newly released community nodes, the system's reliance on custom Python bridging

code can be significantly reduced.

The n8n-nodes-polymarket-gemma-api is a dedicated community node developed specifically for interacting with both the Gamma API (metadata, profiles, searching) and the CLOB API (order creation, orderbook depth).⁴⁷ Installing this node directly into the local n8n instance (via npm or Docker custom extensions) allows the automated workflow to pull market data, pass it through an LLM sentiment filter, and execute orders visually.⁴⁷

Furthermore, n8n's native database integrations allow for the seamless storage of historical price data. Pre-built templates, such as the "Collect historical price data from Polymarket Up/Down markets into Supabase" workflow, automatically convert Unix timestamps, calculate start/end times, and persist the data into query-ready PostgreSQL databases.⁴⁸ This architecture enables offline Python scripts to backtest new Theta and Kelly parameters against historical order book data without hitting live API rate limits.⁴⁸

5. Ranked Matrix of Actionable Short-Term Improvements

The following ranked list outlines the highest-impact improvements that can be implemented within the coming week. They are ordered based on the ratio of expected edge gained to implementation difficulty, strictly adhering to the Python, n8n, and <\$50/month API constraints.

1. Implement Strict Gamma API Liquidity & Volume Filters

- **Implementation Difficulty:** Easy
- **Expected Edge Gain:** High (Immediate elimination of slippage and bid/ask spread losses).
- **Actionable Steps:** Refactor the initial market discovery phase in the Python script. Instead of querying the CLOB, use Python's requests library or an n8n HTTP node to poll https://gamma-api.polymarket.com/events?active=true&closed=false&order=volume_24hr&ascending=false.¹² Write logic to discard any market with less than a hard-coded 24-hour volume threshold (e.g., \$50,000) before any predictive processing occurs.

2. Deploy Fractional Kelly Sizing Logic for Position Management

- **Implementation Difficulty:** Easy
- **Expected Edge Gain:** High (Transforms negative expectancy variance into mathematically protected capital preservation).
- **Actionable Steps:** Provide Claude Code with the mathematical formula for binary Kelly fractions ($f^* = \frac{p(b+1)-1}{b}$).²⁴ Have Claude generate a Python class that ingests the bot's win probability and the current market decimal odds, calculates f^* , multiplies it by a strict 0.25 (Quarter-Kelly) factor, and returns the exact USDC/Mana position size.²⁶ Integrate this

class into the order execution function.

3. Integrate "The Odds API" for Cross-Oracle +EV Polling

- **Implementation Difficulty:** Medium
- **Expected Edge Gain:** High (Unlocks combinatorial arbitrage strategies outside the impossible HFT timeframe).
- **Actionable Steps:** Register for the free tier of The Odds API (500 requests/month).¹⁰ Clone the edge-scanner repository logic.¹⁰ Create an n8n polling workflow that checks major sporting event probabilities twice daily. Use Python to compare the Pinnacle sharp line implied probability against the Polymarket sports contract midpoint.⁹ Execute the trade if the edge delta is greater than 10%.

4. Implement a Local Trailing Stop-Loss (TSL) Loop

- **Implementation Difficulty:** Medium
- **Expected Edge Gain:** Medium/High (Locks in the currently marginal \$17 PnL and prevents winning trades from decaying).
- **Actionable Steps:** Create a stateful Python script (using SQLite) or an n8n database table that tracks open order IDs and their maximum achieved prices.³⁶ Run a cron-triggered script every 15 minutes that fetches current prices via the py-clob-client. If the price drops by a parameterized percentage (e.g., 5%) from the maximum, trigger an automatic /orders POST to market-sell and close the position.¹¹

5. Establish Metaculus Baseline Probability Filters

- **Implementation Difficulty:** Medium
- **Expected Edge Gain:** Medium (Substantially improves trade conviction by mirroring professional human superforecasters).
- **Actionable Steps:** Clone the forecasting-tools Python repository.¹⁶ Use Claude Code to extract the specific functions required to authenticate and pull Superforecaster median probabilities for active questions.¹³ Add a gating check to the trading bot: only permit an entry order if the algorithm's directional bias matches the Metaculus Superforecaster median.⁴⁹

6. Introduce Dynamic Time-Decay (Theta) Hard Exits

- **Implementation Difficulty:** Easy
- **Expected Edge Gain:** Medium (Averts exponential binary gamma risk and terminal wipeouts).
- **Actionable Steps:** Add a timestamp comparison function within the position management script. Identify the precise resolution block of the target market. Using the square-root of time protocol (

$Position(t) = Initial_Position \times \sqrt{T_{remaining} / T_{initial}}$, automatically liquidate positions exactly 24 hours prior to final resolution, completely circumventing post-market Oracle disputes and final-hour crashes.²⁷

7. Install n8n-nodes-polymarket-gemma-api Community Node

- **Implementation Difficulty:** Easy
- **Expected Edge Gain:** Low/Medium (Drastically improves infrastructure stability and reduces custom Python maintenance).
- **Actionable Steps:** Inside the local n8n instance, install the n8n-nodes-polymarket-gemma-api package via the community nodes directory or npm.⁴⁷ Reroute the current Python-based REST requests into these native nodes. This centralizes L2 authentication, signature timestamping, and market querying directly into the visual workflow, reducing code overhead.⁴⁷

8. Set Up Free-Tier News Sentiment Aggregation

- **Implementation Difficulty:** Medium
- **Expected Edge Gain:** Low (Provides macro context without breaking the \$50 budget).
- **Actionable Steps:** Connect NewsAPI.ai or the GNews API to an n8n webhook.²⁰ Feed the headline JSON payloads into a cheap LLM text-classification node (e.g., Claude 3.5 Haiku). If the aggregate sentiment contradicts an open position (e.g., holding "Yes" while the LLM categorizes recent breaking news as entirely negative), immediately lower the TSL threshold to execute a rapid exit.²²

By implementing these structural optimizations over the coming week, the algorithmic architecture will graduate from relying on a fragile 42% win rate to a statistically robust framework characterized by rigorous market selection, mathematical position sizing, and dynamic, capital-protecting exits.

Works cited

1. CarlosIbCu/polymarket-kalshi-btc-arbitrage-bot - GitHub, accessed March 9, 2026, <https://github.com/CarlosIbCu/polymarket-kalshi-btc-arbitrage-bot>
2. How to Programmatically Identify Arbitrage Opportunities on Polymarket (And Why I Built a Portfolio Betting Agent Instead) | by PolyMaster | Medium, accessed March 9, 2026, <https://medium.com/@wanguolin/how-to-programmatically-identify-arbitrage-opportunities-on-polymarket-and-why-i-built-a-portfolio-23d803d6a74b>
3. Manifold Markets Review 2026: The Free Community Prediction Platform - Crypto News, accessed March 9, 2026, <https://cryptonews.com/cryptocurrency/manifold-markets-review/>
4. The Math of Prediction Markets: Binary Options, Kelly Criterion, and CLOB Pricing Mechanics - Navnoor Bawa, accessed March 9, 2026,

- <https://navnoorbawa.substack.com/p/the-math-of-prediction-markets-binary>
5. Beyond Simple Arbitrage: 4 Polymarket Strategies Bots Actually Profit From in 2026, accessed March 9, 2026,
<https://medium.com/illumination/beyond-simple-arbitrage-4-polymarket-strategies-bots-actually-profit-from-in-2026-ddacc92c5b4f>
 6. Sports Betting Bots on Polymarket: Automated Event Trading - QuantVPS, accessed March 9, 2026,
<https://www.quantvps.com/blog/automated-sports-betting-bots-on-polymarket>
 7. Just Found the Math That Guarantees Profit on Polymarket and Why Retail Traders Are Just Providing Liquidity | by Ezekiel Njuguna | Feb, 2026 | Dev Genius, accessed March 9, 2026,
<https://blog.devgenius.io/just-found-the-math-that-guarantees-profit-on-polymarket-and-why-retail-traders-are-just-providing-6163b4c431a2>
 8. Polymarket HFT: How Traders Use AI to Identify Arbitrage and Mispricing - QuantVPS, accessed March 9, 2026,
<https://www.quantvps.com/blog/polymarket-hft-traders-use-ai-arbitrage-mispricing>
 9. betting-strategy · GitHub Topics, accessed March 9, 2026,
<https://github.com/topics/betting-strategy>
 10. Open sourced a simple arb scanner for anyone who wants to play around - Reddit, accessed March 9, 2026,
https://www.reddit.com/r/algobetting/comments/1pkfhvc/open_sourced_a_simple_arb_scanner_for_anyone_who/
 11. Best Prediction Market APIs for Developers and Traders - Forex VPS, accessed March 9, 2026, <https://newyorkcityservers.com/blog/best-prediction-market-apis>
 12. Fetching Markets - Polymarket Documentation, accessed March 9, 2026,
<https://docs.polymarket.com/market-data/fetching-markets>
 13. Bot Tournament Resources Page - Metaculus, accessed March 9, 2026,
<https://www.metaculus.com/notebooks/38928/aib-resource-page/>
 14. Metaculus, accessed March 9, 2026, <https://www.metaculus.com/>
 15. Spring 2026 FutureEval Bot Tournament - Metaculus, accessed March 9, 2026,
<https://www.metaculus.com/tournament/spring-aib-2026/>
 16. Metaculus/forecasting-tools: A framework for building a AI Forecasting Bot for Metaculus. Additionally AI Forecasting tools to help humans forecast the future. - GitHub, accessed March 9, 2026, <https://github.com/Metaculus/forecasting-tools>
 17. Superforecasting AI - Good Judgment, accessed March 9, 2026,
<https://goodjudgment.com/superforecasting-ai/>
 18. Tournaments | Metaculus, accessed March 9, 2026,
<https://www.metaculus.com/tournaments/>
 19. Identifying and Cultivating Superforecasters as a Method of Improving Probabilistic Predictions - Stanford University, accessed March 9, 2026,
<https://web.stanford.edu/~knutson/jdm/mellers15.pdf>
 20. Free News APIs That Work in 2026 [Tested] - Newsdata.io, accessed March 9, 2026, <https://newsdata.io/blog/best-free-news-api/>
 21. Top 5 Best News API solutions in 2026, accessed March 9, 2026,

- <https://www.scrapingbee.com/blog/top-best-news-apis-for-you/>
22. Building a Sentiment-Arbitrage Agent: Polymarket API + Real-Time Forum Search (OpenWebNinja) : r/n8n - Reddit, accessed March 9, 2026,
https://www.reddit.com/r/n8n/comments/1qdopa4/building_a_sentimentarbitrage_agent_polymarket/
 23. Kelly fractions for independent simultaneous bets - Emir's blog, accessed March 9, 2026, <https://emiruz.com/post/2025-01-05-sim-kelly/>
 24. The Kelly Criterion — Hands-On Mathematical Optimization with AMPL in Python, accessed March 9, 2026,
<https://ampl.com/mo-book/notebooks/06/kelly-criterion.html>
 25. How do practitioners approach Kelly formula for portfolio optimization : r/quant - Reddit, accessed March 9, 2026,
https://www.reddit.com/r/quant/comments/10yexm0/how_do_practitioners_approach_kelly_formula_for/
 26. Kelly Criterion: Practical Portfolio Optimization & Allocation - Invest with CARL, accessed March 9, 2026,
<https://investwithcarl.com/learning-center/investment-basics/dynamic-adaptive-kelly-criterion-bridging-theory-and-practice-for-modern-portfolio-optimization>
 27. The Mathematical Execution Behind Prediction Market Alpha: How ..., accessed March 9, 2026,
<https://navnoorbawa.substack.com/p/the-mathematical-execution-behind>
 28. the journal of - Portfolio Management Research, accessed March 9, 2026,
<https://www.pm-research.com/content/iiijfds/5/3/local/complete-issue.pdf>
 29. Mastering the Kelly Criterion for Smarter Crypto Risk Management - LBank, accessed March 9, 2026,
<https://www.lbank.com/explore/mastering-the-kelly-criterion-for-smarter-crypto-risk-management>
 30. Understanding Theta and Time Decay in Options | IBKR Quant, accessed March 9, 2026,
<https://www.interactivebrokers.com/campus/ibkr-quant-news/understanding-the-ta-and-time-decay-in-options/>
 31. Option Theta - Macroption, accessed March 9, 2026,
<https://www.macroption.com/option-theta/>
 32. Application of the Kelly Criterion to Prediction Markets - arXiv.org, accessed March 9, 2026, <https://arxiv.org/html/2412.14144v1>
 33. Time Decay 101: How It Affects Options Trading - Nasdaq, accessed March 9, 2026,
<https://www.nasdaq.com/articles/time-decay-101-how-it-affects-options-trading>
 34. Theta Decay in Options Trading: 3 Strategies | Charles Schwab, accessed March 9, 2026, <https://www.schwab.com/learn/story/theta-decay-options-trading>
 35. GitHub - yllvar/Kalshi-Quant-TeleBot: Kalshi Advanced Quantitative Trading Bot is an enterprise-grade automated trading system designed for the Kalshi event-based prediction market. Built with cutting-edge quantitative algorithms and professional risk management, it provides institutional-quality trading capabilities with user-friendly control, accessed March 9, 2026,

- <https://github.com/yllvar/Kalshi-Quant-TeleBot>
36. Stop-Loss, Take-Profit, Triple-Barrier & Time-Exit: Advanced Strategies for Backtesting, accessed March 9, 2026,
https://medium.com/@jpolec_72972/stop-loss-take-profit-triple-barrier-time-exit-advanced-strategies-for-backtesting-8b51836ec5a2
 37. StephanAkkerman/binance-trailing-stop-loss - GitHub, accessed March 9, 2026,
<https://github.com/StephanAkkerman/binance-trailing-stop-loss>
 38. The Definitive Guide to the Polymarket Ecosystem: 170+ Tools, Bots, and Products Reshaping Prediction Markets, accessed March 9, 2026,
<https://defiprime.com/definitive-guide-to-the-polymarket-ecosystem>
 39. Python client for the Polymarket CLOB - GitHub, accessed March 9, 2026,
<https://github.com/Polymarket/py-clob-client>
 40. Clients & SDKs - Polymarket Documentation, accessed March 9, 2026,
<https://docs.polymarket.com/api-reference/clients-sdks>
 41. vluzko/manifoldpy: Python tools for working with Manifold Markets - GitHub, accessed March 9, 2026, <https://github.com/vluzko/manifoldpy>
 42. discountry/polymarket-trading-bot - GitHub, accessed March 9, 2026,
<https://github.com/discountry/polymarket-trading-bot>
 43. Automate trading on Polymarket with this beginner-friendly Python bot featuring gasless transactions and real-time data updates. - GitHub, accessed March 9, 2026, <https://github.com/Panca2341/polymarket-trading-bot>
 44. OpenClaw x Polymarket Analysis: Understanding Automated Trading Strategies on Phemex (2026 Update), accessed March 9, 2026,
<https://phemex.com/blogs/openclaw-polymarket-automated-trading-analysis>
 45. OpenClaw Polymarket Bot: Automate Trading in 2026 - Flypixon, accessed March 9, 2026, <https://flypixon.ai/openclaw-polymarket-trading/>
 46. OpenClaw — Personal AI Assistant - GitHub, accessed March 9, 2026,
<https://github.com/openclaw/openclaw>
 47. n8n-nodes-polymarket-gemma-api 0.0.3 on npm - Libraries.io, accessed March 9, 2026, <https://libraries.io/npm/n8n-nodes-polymarket-gemma-api>
 48. Collect historical price data from Polymarket Up/Down markets into Supabase - N8N, accessed March 9, 2026,
<https://n8n.io/workflows/12823-collect-historical-price-data-from-polymarket-up-down-markets-into-supabase/>
 49. Comparing Forecasting Track Records for AI Benchmarking and Beyond - Metaculus, accessed March 9, 2026,
<https://www.metaculus.com/notebooks/28552/comparing-forecasting-track-records-for-ai-benchmarking-and-beyond/>
 50. The Truth About ODTE Options Time Decay, accessed March 9, 2026,
<https://optionalpha.com/blog/Odte-options-time-decay>
 51. n8n Automates Analysis of Polymarket Events with GPT and Nansen - Phemex, accessed March 9, 2026,
<https://phemex.com/news/article/n8n-automates-analysis-of-polymarket-events-with-gpt-and-nansen-48013>